

Практическая работа №3

Изучение работы с системой контроля версий

1 Цель работы

- 1.1 Познакомиться с назначением системы контроля версий Git;
- 1.2 Научиться создавать локальный репозиторий, фиксировать изменения и просматривать историю через графический интерфейс Visual Studio;
- 1.3 Научиться выполнять основные операции Git через консоль: проверять состояние, индексировать изменения, создавать коммиты, работать с ветками и объединять их..

2 Литература

2.1 Тракимус, Ю. В. Разработка консольных приложений с помощью Microsoft Visual Studio 2017: учебное пособие / Ю. В. Тракимус. – Новосибирск : Новосибирский государственный технический университет, 2018. – URL: <https://ibooks.ru/reading.php?productid=367816>. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – с.4-37.

3 Подготовка к работе

- 3.1 Повторить теоретический материал (см. п.2).
- 3.2 Изучить описание лабораторной работы.

4 Основное оборудование

- 4.1 Персональный компьютер.

5 Задание

- 5.1 Создание проекта в Visual Studio

Скриншоты всех шагов создания проекта вставьте в отчет

- 5.1.1 Запустить Microsoft Visual Studio и создать новый проект

Консольное приложение (Майкрософт) на языке C#.

- 5.1.2 Указать имя проекта «GitPractice»

- 5.1.3 Открыть Program.cs и заменить содержимое на приведенный код:

```
Console.Write("Введите ваше имя: ");  
string? name = Console.ReadLine();  
Console.WriteLine($"Здравствуйте, {name}!");
```

- 5.1.4 Запустить проект. Ввести имя и проверить результат работы программы. Скриншот работающей программы **вставить в отчет.**

- 5.2 Создание локального репозитория через Visual Studio

- 5.2.1 В Visual Studio в меню **Git** выбрать опцию **Создать репозиторий Git**

- 5.2.2 В окне создания репозитория выбрать вариант «Локальный» и нажать кнопку **Создать**

- 5.2.3 В меню **Вид** выбрать опцию **Репозиторий Git**. В открывшемся окне дважды кликнуть на самый верхний коммит и изучить содержимое открывшейся вкладки. Скриншот с информацией данного коммита **вставить в отчет.**

5.2.4 Открыть файл Program.cs и добавить обработку пустого имени между вводом и строкой приветствия

```
if (string.IsNullOrEmpty(name))  
{  
    name = "пользователь";  
}
```

5.2.5 Открыть вкладку **Изменения Git** и дважды кликнуть на файл в списке изменений. Изучить открывшуюся вкладку со сравнением изменённых строк. Скриншот вкладки **вставить в отчет**.

5.2.6 Во вкладке **Изменения Git** нажать на символ «+» напротив измененного файла. Ввести осмысленное сообщение коммита, описывающее изменения в коде в поле ввода и нажать **Зафиксировать промежуточные элементы**.

5.2.7 В меню **Вид** выбрать опцию **Репозиторий Git**. В истории должен появиться новый коммит с указанным сообщением. Скриншот с историей коммитов **вставить в отчет**

5.3 Проверка репозитория через консоль

5.3.1 Правой клавишей нажать на имя решения в **Обозревателе решений**, и в контекстном меню выбрать пункт **Открыть в терминале**. Либо в меню **Вид** выбрать пункт **Терминал**.

5.3.2 Убедиться, что терминал открылся в корневой папке решения

5.3.3 Выполнить команду `git status`. Команда должна сообщить, что выбрана ветка master и рабочее древо чистое. Если же имеются изменения определить их источник и отменить или зафиксировать изменения в репозитории.

5.3.4 Просмотреть краткую историю с графом и указателями веток при помощи команды:

```
git log --oneline --graph --decorate -all
```

5.3.5 Скопировать **в отчет** вывод команд status и log

5.4 Создание коммита через консоль

5.4.1 Создать в корне решения файл README.md и открыть его в любом текстовом редакторе. Записать название работы, имя студента и краткое назначение программы.

5.4.2 Открыть терминал в корневой папке решения. Проверить состояние репозитория при помощи команды:

```
git status
```

Новый файл должен отображаться как untracked, то есть неотслеживаемый

5.4.3 Добавить README.md в индекс и проверить результат:

```
git add README.md  
git status  
git diff --cached
```

Команда `diff --cached` показывает именно индексированные изменения.

Результат выполнения команд занести **в отчет**

5.4.4 Создать коммит и снова проверить состояние:

```
git commit -m "Добавлено описание проекта"  
git status  
git log --oneline -3
```

Результат выполнения команд занести **в отчет**

5.5 Работа с веткой через Visual Studio

5.5.1 В меню **Git** выбрать опцию **Создать ветвь**. В диалоговом окне указать имя ветви «feature-greeting» и нажать **Создать**

5.5.2 Проверить в строке состояния (справа снизу), что активна ветка feature-greeting. Не вносить изменения, пока это имя не отображается как текущая ветка.

5.5.3 Изменить Program.cs: изменить текст в сообщении приветствия. Запустить программу и проверить вывод.

5.5.4 В окне **Изменения Git** просмотреть разницу, ввести осмысленное сообщение коммита и зафиксировать все изменения.

5.5.5 В меню **Вид** выбрать опцию **Репозиторий Git**. В списке локальных ветвей должна быть feature-greeting, а граф истории должен показывать новый коммит только в этой ветке.

5.5.6 В окне **Изменения Git** переключиться обратно на ветку master. Проверить, что изменения в Program.cs пропали.

5.5.7 В окне **Репозиторий Git** щелкнуть правой кнопкой feature-greeting и выбрать «Объединить feature-greeting с master». Подтвердить операцию

5.5.8 Открыть Program.cs и убедиться, что изменения строки приветствия появились в master. Просмотреть граф истории и сохранить снимок результата слияния **в отчет**.

5.6 Разрешение конфликта изменений

5.6.1 Открыть терминал в корневой папке решения. В терминале создать ветку conflict-demo и перейти в нее:

```
git switch -c conflict-demo
```

5.6.2 В README.md найти строку с кратким описанием проекта и заменить ее на «Учебный проект для изучения ветвления Git». Сохранить и зафиксировать изменение:

```
git add README.md  
git commit -m "Изменено описание в учебной ветке"
```

5.6.3 Переключиться на master:

```
git switch master
```

5.6.4 В той же строке README.md записать другой текст: «Консольное приложение для практики Git». Сохранить и зафиксировать изменение:

```
git add README.md  
git commit -m "Уточнено описание проекта"
```

5.6.5 В Visual Studio Открыть окно «Репозиторий Git». Щелкнуть правой кнопкой на conflict-demo и выбрать объединение этой ветки с master. Git должен остановить слияние и показать README.md в группе

Необъединенные изменения в окне Изменения Git.

5.6.6 Дважды щелкнуть на конфликтующий файл и открыть редактор слияния. Сравнить текущий вариант main и входящий вариант conflict-demo. В области результата вручную записать согласованный текст: «Консольный учебный проект для практики ветвления Git».

5.6.7 Удалить маркеры конфликта <<<<<<, ===== и >>>>>>, если они есть в редакторе. Итоговый файл не должен содержать эти строки.

5.6.8 Нажать **Принять слияние**. В окне **Изменения Git** ввести сообщение «Разрешен конфликт описания проекта» и создать коммит слияния.

5.6.9 Открыть окно **Репозиторий Git** и изучить граф истории после слияния. Скриншот графа истории вставьте **в отчет**

5.7 Отмена коммита

5.7.1 Добавить в README.md строку «Временная заметка», создать отдельный коммит с сообщением «Добавлена временная заметка»

5.7.2 В окне **Репозиторий Git** щелкнуть этот коммит правой кнопкой и выбрать **Отменить**.

5.7.3 Убедиться, что Visual Studio создала новый коммит, который удаляет временную строку из файла, но сохранила исходный коммит в истории. Открыть окно **Репозиторий Git** и изучить граф истории после отмены коммита. Скриншот графа истории вставьте **в отчет**

6 Порядок выполнения работы

6.1 Используя Microsoft Visual Studio и терминал выполнить задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Снимки экрана и результаты выполнения команд по ходу выполнения работы

7.4 Ответы на контрольные вопросы

7.5 Вывод

8 Контрольные вопросы

8.1 Чем Git отличается от GitHub?

8.2 Как определить текущую ветку и состояние файлов в Visual Studio?

8.3 Как определить текущую ветку и состояние файлов через консоль?

8.4 Почему возникает конфликт слияния?

9 Приложение

9.1 Таблица основных команд Git

Команда	Назначение
git status	Показать текущую ветку и состояние файлов

Команда	Назначение
<code>git status --short</code>	Показать состояние в компактном виде
<code>git diff</code>	Показать незаписанные в индекс изменения
<code>git add <файл></code>	Добавить выбранный файл в индекс
<code>git diff --cached</code>	Показать подготовленные к коммиту изменения
<code>git commit -m "..."</code>	Создать коммит с сообщением
<code>git log --oneline --graph --all</code>	Показать компактный граф истории всех ветвей
<code>git switch -c <ветка></code>	Создать ветку и перейти в нее
<code>git switch <ветка></code>	Переключиться на существующую ветку
<code>git merge <ветка></code>	Объединить указанную ветку с текущей
<code>git restore <файл></code>	Отменить незакоммиченные изменения файла
<code>git revert <коммит></code>	Создать новый коммит, отменяющий выбранный
<code>git remote -v</code>	Показать адреса удаленных репозиториях
<code>git push -u origin main</code>	Впервые отправить main и настроить отслеживание